1. Develop a C program that demonstrates how a Linux operating system executes a command entered by a user that 1. Accept a Linux command as input. 2. Create a child process using fork(). 3. Execute the command in the child process using an appropriate exec() system call. 4. Allow the parent process to wait for the child using wait (). 5. Display the Process ID (PID) of both parent and child processes.

Using Linux terminal commands (uname, lscpu, lsblk, ps, top), investigate the relationship between hardware resources and operating system services. Prepare a report explaining how the OS abstracts CPU, memory, storage, and I/O devices.

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
int main()
{
    char command[100];
C

printf("Enter a Linux command: ");
scanf("%s", command);

pid_t pid = fork();

if(pid == 0)
{
    printf("\nChild Process\n");
    printf("Child PID = %d\n", getpid());

    execlp(command, command, NULL);

    printf("Command execution failed!\n");
}
else if(pid > 0)
{
    printf("\nParent Process\n");
    printf("Parent PID = %d\n", getpid());

    wait(NULL);

    printf("Child process completed.\n");
}
else
{
    printf("Fork failed!\n");
}

return 0;
}
```

```
thumu_hemanth_kumar@Hemanths-TUF-Laptop:~$ nano practical1.c
thumu_hemanth_kumar@Hemanths-TUF-Laptop:~$ gcc practical1.c -o practical
thumu_hemanth_kumar@Hemanths-TUF-Laptop:~$ ./practical
Enter a Linux command: ls

Parent Process
Parent PID = 805

Child Process
Child PID = 806
OSPractical  practcal1.c  practical     practical2    program1    program2.c.save
os           practial1    practical1.c  practical2.c  program1.c
Child process completed.
```